

Aufgabe 9

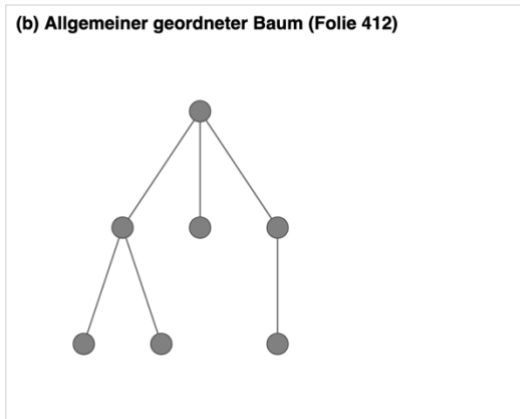
Aufgabe 9.1

Erklärung:

Zwei Bäume aus den Folien werden mit dem Walker-Algorithmus (Hierarchy.tidy aus gampleman/elm-visualization) gelayoutet: der binäre Baum von Folie 405 (A–O) und der allgemeine geordnete Baum von Folie 412 (root mit home, etc, var). Beide sind als Tree String aus gampleman/elm-rosetree modelliert. Gezeichnet wird per TypedSvg: Kanten als graue Linien (via Tree.links), Knoten als kleine graue Kreise. Über `Html.Events.onMouseOver / onMouseOut` wird ein `hovered : Maybe String` im Model gesetzt – der gehoverte Kreis bekommt seine Knotenfarbe (deterministisch aus `Char-Code-Summe mod 8-Palette`) und darüber erscheint ein Label im selben Farbton. Unter den Visualisierungen folgt die textuelle Ausgabe: alle (Kind, Maybe Elternteil)-Paare und die berechneten Layout-Koordinaten (x, y) als Gitter (x relativ zur Wurzel, y = Tiefe).

Screenshots:

(b) Allgemeiner geordneter Baum (Folie 412)



Converted testTree (Child, Maybe Parent)

- (root, Nothing)
- (home, root)
- (user1, home)
- (user2, home)
- (etc, root)
- (var, root)
- (log, var)

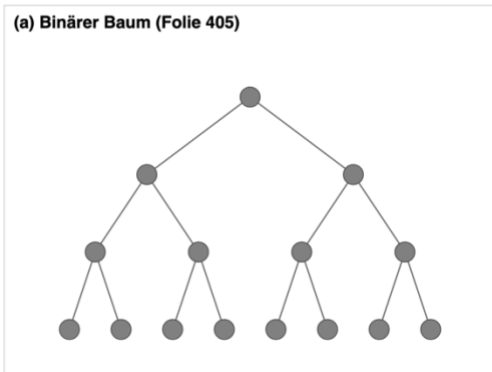
Tree Layout

- (etc, x=1,y=3)
- (home, x=-1,y=3)
- (log, x=3,y=4)
- (root, x=1,y=1)
- (user1, x=-2,y=4)
- (user2, x=0,y=4)
- (var, x=3,y=3)

Übung 9.1 – Explizite Baumdarstellungen (Walker-Algorithmus)

Layout berechnet mit Hierarchy.tidy aus gampleman/elm-visualization. Bewege die Maus über einen Knoten, um den Namen einzublenden.

(a) Binärer Baum (Folie 405)



Link: <https://ellie-app.com/yPQ3gXmbGNya1>

Aufgabe 9.2

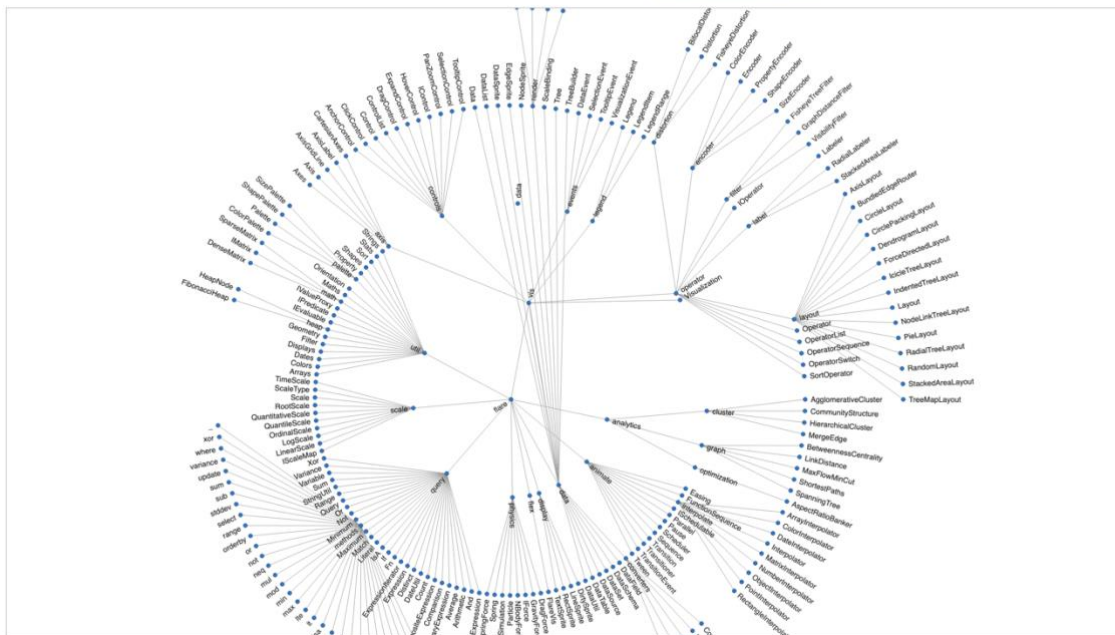
Erklärung:

Der Flare-Baum wird aus flare.json per Http.get und rekursivem Json.Decode geladen. Layoutiert wird er mit Hierarchy.tidy aus gampleman/elm-visualization; die so erhaltenen kartesischen Koordinaten werden anschließend polar transformiert ($x \rightarrow \alpha = (x - x_{\min}) / \Delta x \cdot 2\pi$, $y \rightarrow r = \text{Tiefe} / \text{maxTiefe} \cdot R$), sodass die Wurzel im Mittelpunkt sitzt und die Blätter den äußeren Ring bilden – wie in der D3-Vorlage radial-tree-component. Gezeichnet wird per TypedSvg (Linien für Kanten, kleine Kreise für Knoten); die Beschriftungen werden um ihren Winkel rotiert und auf der linken Halbkreishälfte zusätzlich um 180° gespiegelt, damit der Text immer von innen nach außen lesbar bleibt.

Screenshot:

Übung 9.2 – Flare als radialer Baum

Daten: flare.json. Layout per Hierarchy.tidy (x → Winkel, y → Radius), radiale Anordnung wie in [D3 radial-tree-component](#). Labels stehen radial nach außen.



Link: <https://ellie-app.com/yPRPML4NySVa1>

Aufgabe 9.3

Erklärung:

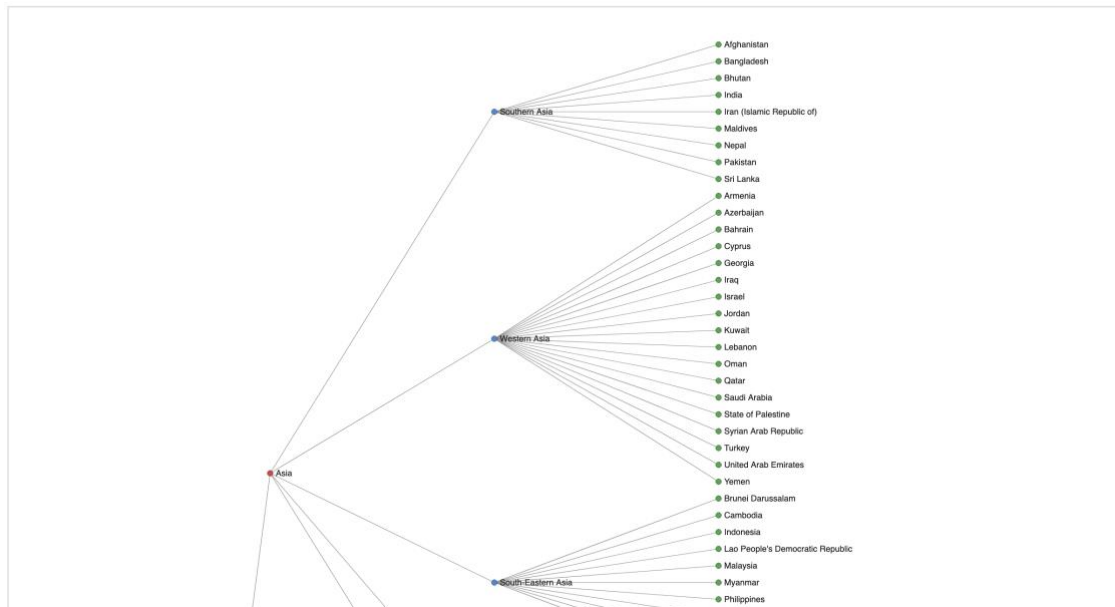
Aus countryHierarchy.json wird die Welt → Kontinente → Subregionen → Länder-Struktur per Http.get geladen. Ein eigener countryDecoder liest data.id als Knoten-

Label und ruft sich für das optionale children-Array rekursiv via D.lazy auf — das Ergebnis ist ein Tree String aus gampleman/elm-rosetree. Das Layout berechnet Hierarchy.tidy aus gampleman/elm-visualization (Walker-Algorithmus); die kartesischen Koordinaten werden durch Vertauschen von x und y in eine horizontale Anordnung überführt, sodass die Tiefe nach rechts wächst und die ca. 200 Länder-Namen waagerecht lesbar untereinander stehen. Gezeichnet wird per TypedSvg (Linien für Kanten, Kreise für Knoten, ebenen-abhängige Farben).

Screenshot:

Übung 9.3 – Länder-Hierarchie

Daten: countryHierarchy.json (World → Region → Subregion → Land). Eigener Decoder auf data.id, Layout per Hierarchy.tidy aus gampleman/elm-visualization.



Link: <https://ellie-app.com/yPS62pWV3t2a1>